

ICPC Prep — Week 2

1 GCD & LCM (Euclidean Algorithm)

Idea

The **greatest common divisor** $\gcd(a, b)$ is the largest integer dividing both a and b ; it shows up constantly in number-theory problems. **Euclid's algorithm** computes it from one fact: any common divisor of a and b also divides $a - b$, so subtracting (or, faster, taking a remainder) never changes the gcd. Repeating this shrinks the numbers until one hits 0:

$$\gcd(a, b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b > 0 \end{cases}$$

Key facts

- Running time is $O(\log \min(a, b))$.
- $\gcd(a, 0) = a$, and $\gcd(a, b)$ is the smallest positive value of $ax + by$ over integers x, y (Bézout). This is the hook for the modular inverse below.
- **LCM via GCD:** $\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$. In code, divide *before* multiplying — `a / gcd(a, b) * b` — so the intermediate product cannot overflow.
- Careful: in C++, `a % b` takes the sign of `a` (`-5 % 3 == -2`).

Implementation (C++)

```
1 long long gcd(long long a, long long b) { return b ? gcd(b, a % b) : a; }
2 long long lcm(long long a, long long b) { return a / gcd(a, b) * b; }
3 // C++17 also provides std::gcd(a, b) and std::lcm(a, b) in <numeric>.
```

Extended Euclid & modular inverse

Every remainder Euclid produces is an integer combination of a and b , so alongside $g = \gcd(a, b)$ we can also recover x, y with $ax + by = g$. When $\gcd(a, m) = 1$ this gives $ax + my = 1$, i.e. $ax \equiv 1 \pmod{m}$ — so x is the **modular inverse** of a modulo m . This works for *any* modulus coprime to a , and connects directly to the next section (where a prime modulus lets us use Fermat instead).

```

1 // Returns g = gcd(a, b) and sets x, y so that a*x + b*y = g.
2 long long ext_gcd(long long a, long long b, long long &x, long long &y) {
3     if (b == 0) { x = 1; y = 0; return a; }
4     long long x1, y1;
5     long long g = ext_gcd(b, a % b, x1, y1);
6     x = y1;
7     y = x1 - (a / b) * y1;
8     return g;
9 }
10
11 // Modular inverse of a mod m, valid when gcd(a, m) == 1.
12 long long inverse(long long a, long long m) {
13     long long x, y;
14     ext_gcd(a, m, x, y);
15     return (x % m + m) % m; // normalise into [0, m)
16 }

```

Practice

- [Codeforces 664A — Complicated GCD](#)
- [HackerRank — Sherlock and GCD](#)
- [LeetCode — Nth Magical Number \(LCM\)](#)
- [LeetCode — Ugly Number III \(LCM\)](#)

References

[cp-algorithms — Euclidean algorithm](#) · [cp-algorithms — Extended Euclidean algorithm](#)

Video

[The extended Euclidean algorithm in one simple idea](#)

2 Binomial Coefficients modulo a Prime

Idea

The **binomial coefficient** $\binom{n}{k}$ counts the ways to choose k objects out of n distinguishable ones; equivalently it is the coefficient of $x^k y^{n-k}$ in the expansion of $(x + y)^n$. It appears everywhere in combinatorics, and

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Contest answers are usually enormous, so problems ask for the value modulo a large prime, typically $p = 10^9 + 7$. We cannot divide directly modulo p ; instead each division becomes a multiplication by a **modular inverse**, computed with fast exponentiation (Week 1).

Key facts

- **Pascal's rule:** $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, with $\binom{n}{0} = \binom{n}{n} = 1$. This builds Pascal's triangle with additions only — no division — and is the simplest method when n is small or there is no modulus.
- **Symmetry:** $\binom{n}{k} = \binom{n}{n-k}$.
- **Inverse via Fermat:** since p is prime, $x^{-1} \equiv x^{p-2} \pmod{p}$. So

$$\binom{n}{k} \equiv n! \cdot (k!)^{-1} \cdot ((n-k)!)^{-1} \pmod{p}.$$

- **Precompute once:** build $\text{fact}[i] = i!$ and its inverses $\text{inv_fact}[i]$ in $O(n)$, then answer each $\binom{n}{k}$ in $O(1)$. Only one exponentiation is needed: invert the top factorial, then walk down with $\text{inv_fact}[i-1] = \text{inv_fact}[i] \cdot i$.

Implementation (C++)

```
1 const long long MOD = 1000000007LL;
2 const int MAXN = 1000000;
3 long long fact[MAXN + 1], inv_fact[MAXN + 1];
4
5 // (a^n) mod m in O(log n) -- same primitive as Week 1.
6 long long binpow_mod(long long a, long long n, long long m) {
7     a %= m;
8     long long res = 1 % m;
9     while (n > 0) {
10         if (n & 1) res = res * a % m;
11         a = a * a % m;
12         n >>= 1;
13     }
14     return res;
15 }
16
17 // Precompute factorials and their inverses, then answer each C(n,k) in O(1).
18 void build_factorials() {
19     fact[0] = 1;
20     for (int i = 1; i <= MAXN; ++i) fact[i] = fact[i - 1] * i % MOD;
21     inv_fact[MAXN] = binpow_mod(fact[MAXN], MOD - 2, MOD); // one inverse via Fermat
22     for (int i = MAXN; i > 0; --i)
23         inv_fact[i - 1] = inv_fact[i] * i % MOD; // then walk downward
24 }
25
26 // n choose k, modulo MOD.
27 long long C(int n, int k) {
28     if (k < 0 || k > n) return 0;
29     return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
30 }
```

Practice

- [POJ 2084 — Game of Connections](#) (the n th Catalan number $\binom{2n}{n}/(n+1)$; needs big integers)

- [CSES — Binomial Coefficients](#) (direct $\binom{a}{b} \bmod 10^9+7$ — exactly the code above)
- [CSES — Distributing Apples](#) (stars and bars: m apples to n children is $\binom{n+m-1}{m}$)

References

[cp-algorithms — Binomial Coefficients](#)

Video

[Binomial Coefficients](#)

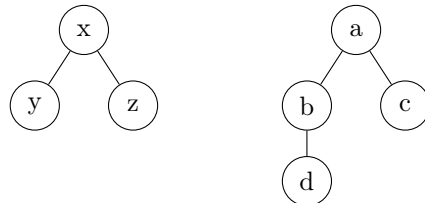
3 Union-Find (Disjoint Sets)

Idea

A **union-find** (disjoint-set) structure stores a collection of disjoint sets and supports two operations, both in essentially constant time:

- **Find**(x) — return the *representative* of the set containing x ;
- **Union**(x, y) — merge the two sets containing x and y .

Represent each set as a rooted tree: every element links to a parent, and the root is the set's representative. Two elements are in the same set exactly when **Find** returns the same root.



Two disjoint sets stored as rooted trees: $\{x, y, z\}$ and $\{a, b, c, d\}$. Each root (x and a) is its set's representative.

Key facts

- **Find** walks parent links up to the root; **Union** finds both roots and points one at the other.
- Naively the trees can grow deep. **Path compression** fixes this: after **Find** locates the root, it re-points every node on the path straight to the root, so later queries are near-instant. (We only care that the root stays the same.)
- With path compression the amortised cost per operation is almost constant, and the whole implementation is only a few lines.

Algorithm

1. Initialise: every element is its own parent (its own one-element set).
2. **Find**(x): follow parent links until a node points to itself — that is the root — then compress the path back down to it.
3. **Union**(x, y): set the parent of **Find**(x) to **Find**(y).

Implementation (C++)

```
1 const int MAXV = 100000;
2 int par[MAXV];
3
4 void init(int n) { for (int i = 0; i < n; ++i) par[i] = i; }
5
6 // Find with path compression: re-points nodes straight to the root.
7 int find(int x) { return x == par[x] ? x : par[x] = find(par[x]); }
8
9 // Merge the sets containing x and y.
10 void unite(int x, int y) { par[find(x)] = find(y); }
11
12 // same_set(x, y): find(x) == find(y)
```

Practice

- [POJ 1988 — Cube Stacking](#) (union-find with extra info along the links; path compression)
- [POJ 1703 — Find them, Catch them](#) (two-group / weighted union-find)
- [POJ 1984 — Navigation Nightmare](#) (union-find with coordinates)

References

[cp-algorithms — Disjoint Set Union](#)

Video

[Disjoint Set \(Union-Find\): union by rank & path compression](#)